UNIVERSITY OF INFORMATION TECHNOLOGY AND SCIENCES

Department of Computer Science and Engineering

CAMPUSCONNECT

University Club, Event & Activity Management Platform

PROJECT DOCUMENTATION

Implementation, Architecture, API, Database and Testing Guide

Course	Software Project Design and Development
Course Code	416
Submitted By	Md. Shakhawat Hossain
Student ID	0432320005101009
Batch	54
Submitted To	Al Imtiaz, Course Teacher
Department	Computer Science and Engineering
Institution	University of Information Technology and Sciences
Version	1.0
Date	27 October 2026

Prepared from the supplied CampusConnect SRS and Final Diagrams for academic project implementation.

1. Executive Summary

CampusConnect is a web-based university club, event and activity management platform. It centralizes event discovery, event registration, club membership requests, event approval, attendance management, announcements, participation certificates and scoped activity reporting. This documentation describes the supplied requirements and the runnable academic implementation included with this repository.

Selected implementation stack: React + TypeScript + Tailwind CSS frontend; Node.js + Express REST API; PostgreSQL database managed by Prisma; JWT authentication; bcrypt password hashing; Zod validation; QR token generation; and PDFKit certificate output.

2. Project Scope and Roles

Role	Core responsibilities
Student	Browse published events and clubs, register for events, request club membership, view own registrations and download own certificates.
Club Executive	Manage events for assigned clubs, submit event proposals, manage member requests, record attendance and publish announcements.
Faculty Advisor	Review event proposals for assigned clubs and view scoped activity information.
University Administrator	Platform-level administration, review, and reporting.

Initial release exclusions

• Online payments, ticket sales and refunds.

• Native Android/iOS applications.

• Biometric attendance and location tracking.

• Official university SSO/SIS integration.

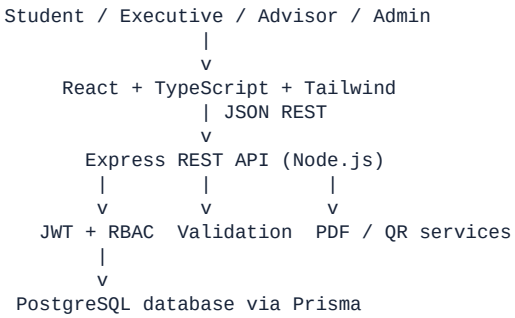
• Automated disciplinary decisions and public student rankings.

3. Requirements Traceability

SRS area	Implementation mapping
FR-01–FR-08: accounts and security	Student registration, login, JWT, password hashing, role checks and protected records.
FR-09–FR-16: events and registration	Published event discovery/search, event detail, registration, deadline/capacity/duplicate checks and own registration history.
FR-17–FR-21: clubs	Active club directory and membership requests with executive review endpoint.
FR-22–FR-28: event approval	Event draft creation, submission, advisor/admin decision, approval log and publication gate.
FR-29–FR-37: attendance/certificates	Attendee listing, manual/QR-token attendance endpoint, PDF certificate generation for attendees and announcements.
FR-38–FR-42: reporting	Scoped summary counts and role-scoped access; advanced export remains a future enhancement.
NFR-01–NFR-12	Hashing, server-side authorization, environment secrets, relational constraints, modular structure, responsive UI and test plan.

4. System Architecture

The system follows a three-tier design. The browser UI calls the Express API with JSON. The API validates requests, authenticates users, checks role and resource ownership, and uses Prisma to read/write PostgreSQL. QR tokens and certificate PDFs are generated by backend services. Production traffic must use HTTPS.



Event approval lifecycle

Draft → **Submitted** → **Approved** → **Published** → **Completed**. Alternative states include Rejected, Changes Requested and Cancelled. Only published events are shown in the public event directory. An advisor or administrator must record the review decision before publication.

5. Repository Architecture

```
CampusConnect/
■■■ client/
■   ■■■ src/
■       ■■■ components/     Modal and shared UI
■       ■■■ lib/             API client and session helpers
■       ■■■ pages/           Login, Dashboard, Events, Clubs, MyRecords
■       ■■■ App.tsx          App shell and role-aware navigation
■       ■■■ main.tsx         React entry point
■■■ server/
■   ■■■ prisma/              schema.prisma and seed.ts
■   ■■■ src/
■       ■■■ middleware/      Authentication and errors
■       ■■■ routes/          Auth, events, clubs, misc endpoints
■       ■■■ services/        PDF certificate service
■       ■■■ app.ts           Express app
■       ■■■ index.ts         API bootstrap
■■■ docs/                    Technical documentation and source PDFs
■■■ docker-compose.yml       Local PostgreSQL
■■■ .env.example             Environment template
■■■ README.md                Quick start and GitHub steps
```

6. Local Installation and Run Instructions

• Install Node.js 20 or newer, npm 10 or newer, and Docker Desktop or PostgreSQL.

• Copy `.env.example` to `.env`; change JWT_SECRET to a random secret of at least 32 characters.

• Start PostgreSQL: `docker compose up -d postgres`.

• Install packages: `npm install`.

• Generate Prisma client: `npm run db:generate`.

• Apply database schema: `npm run db:migrate` (if prompted, use `npm run db:migrate -- --name init`).

• Seed demonstration users and sample club/event: `npm run db:seed`.

• Run client and API: `npm run dev`.

• Open `http://localhost:5173`; API health check is `http://localhost:4000/api/health`.

Demo accounts

Role	Email	Password
Administrator	admin@campusconnect.local	CampusConnect123!
Faculty Advisor	advisor@campusconnect.local	CampusConnect123!
Club Executive	executive@campusconnect.local	CampusConnect123!
Student	student@campusconnect.local	CampusConnect123!

Security warning: these accounts are for local demonstrations only. Do not use these credentials in a public deployment.

7. API Overview

Method / route	Access	Purpose
POST /api/auth/register	Public	Create student account
POST /api/auth/login	Public	Authenticate and receive JWT
GET /api/events	Public	Published events and search
POST /api/events	Executive/Admin	Create event draft for authorized club
POST /api/events/:id/submit	Owner/Admin	Submit proposal for review
POST /api/events/:id/decision	Advisor/Admin	Approve, reject or request changes
POST /api/events/:id/publish	Owner/Admin	Publish approved event
POST /api/events/:id/registrations	Student	Register and receive QR token
GET /api/me/registrations	Student	View own registrations
GET /api/clubs	Public	List active clubs
POST /api/clubs/:id/membership-requests	Student	Request membership
POST /api/events/:id/attendance	Owner/Admin	Record attendance
POST /api/events/:id/certificates	Owner/Admin	Issue PDF certificate records
GET /api/me/certificates	Student	List own certificates
GET /api/reports/summary	Scoped staff	Summary counts
GET /api/health	Public	API liveness

For request bodies, response examples, and complete endpoint details, see `docs/API.md`. Protected endpoints require `Authorization: Bearer `.

8. Database Model and Integrity

The relational schema contains User, StudentProfile, Club, ClubMember, EventCategory, Event, Registration, Attendance, Certificate, Announcement, Notification, Feedback and ApprovalLog. Foreign keys preserve relationships; unique constraints protect email, membership pairs, event/user registration pairs, attendance per registration and certificate codes.

• Normalize email addresses to lowercase before storage.

• Do not permit duplicate active registrations or duplicate attendance.

• Only eligible attendees should receive certificates.

• Keep approval actor, decision and timestamp in ApprovalLog.

• Use UTC timestamps in production and render in the intended local timezone.

• Prefer archiving/deactivation to deleting records needed for audit history.

9. Security and Privacy

• Passwords are stored as bcrypt hashes, never plaintext.

• JWT is verified on every protected request and has an expiry.

• API middleware enforces role permissions; event and club endpoints check ownership.

• Zod validates input and API errors avoid returning stack traces to clients.

• Secrets belong in environment configuration, never in GitHub.

• Production requires HTTPS, secret rotation, strict CORS, backups, rate limiting, monitoring and a security review.

• Collect only necessary personal information and obtain university approval for real student records.

10. Testing and Acceptance

Test ID	Scenario	Expected result
TC-01	Register with valid details	Account created; duplicate email rejected
TC-02	Invalid login	Safe authentication error
TC-03	Student accesses restricted route	Unauthorized/forbidden response
TC-04	Register for open event	One registration and QR token created
TC-05	Duplicate registration	Duplicate prevented
TC-06	Full/closed event	Request rejected with reason
TC-07	Executive edits another club event	Access denied
TC-08	Advisor reviews submitted event	Decision and audit log saved
TC-09	Mark attendance with valid token	Attendance recorded once
TC-10	Reuse attendance token/record	Duplicate rejected
TC-11	Download certificate	Owner receives PDF; unrelated user denied
TC-12	Mobile viewport	Main screens remain usable
TC-13	Invalid dates/capacity	Validation rejects input
TC-14	Unrelated report access	Access denied or scope enforced
TC-15	Restart application	Committed data persists

Run `npm test` for automated smoke tests and `npm run build` for build/type checks. Test outcomes must be recorded only after actually running them; this report does not claim all acceptance tests have passed.

11. Known Limitations and Future Work

• Email provider is not configured; external email delivery and retries require provider credentials and implementation.

• QR token entry is supported by the API, but browser camera scanning UI is not included.

• Production hosting, domain, TLS, backup restoration, performance testing and penetration testing are not configured.

• Account recovery, advanced analytics/export, audit UI, and optional feedback UI are future enhancements.

• University SSO/SIS, native mobile apps, payments, biometrics and location tracking are outside initial scope.

12. GitHub Upload

```
git init
git add .
git commit -m "Initial CampusConnect implementation"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/YOUR-REPOSITORY.git
git push -u origin main
```

Create an empty repository on GitHub, replace the remote URL with your repository URL, and run these commands from the project root. Do not commit `.env`, tokens, database backups or real student records.

13. Source Documents and Design Basis

This implementation was based on the supplied CampusConnect SRS Version 1.0 draft dated 13 October 2026 and the supplied Final Diagrams assignment. Original source PDFs are retained under `docs/reference/`. The supplied diagrams identify four main roles, relational entities, event registration validation and event approval lifecycle. This report records the selected implementation choice (PostgreSQL) and highlights remaining deployment work rather than treating optional SRS items as already configured.

Prepared for: Software Project Design and Development, Course Code 416. Submitted by Md. Shakhawat Hossain, Student ID 0432320005101009, Batch 54. Submitted to Al Imtiaz, Course Teacher.